

Serviceauftrag

Glauser Illnau AG – Sanitärunternehmen

Projektdokumentation

Teil 1: Analyse und Entwurf · Teil 2: Implementierung

Autoren	Mykhailo Khimich, Nikolay Riva, Florian Elser
Klasse	IMST24a – BWZ Rapperswil-Jona
Modul	Webanwendung (Spring Boot / React)
Datum	23. Juni 2026
Version	1.0

Inhaltsverzeichnis

Inhaltsverzeichnis	2
1 Beschreibung der Anwendung	5
1.1 Prozessablauf.....	5
1.2 Funktionale Anforderungen.....	5
1.3 Architektur, Technologien und Schnittstellen	5
1.4 Technologieentscheidung: React vs. Thymeleaf	5
2 OO-Modellierung.....	7
2.1 UML-Zustandsdiagramm (Navigation).....	7
2.2 Anwendungsfalldiagramm (Use-Cases & Aktoren).....	7
2.3 Klassendiagramm	9
3 ERM / ERD für die PostgreSQL-Datenbank	10
3.1 Verfeinertes Datenmodell (Attribute, Datentyp, Wertebereich).....	10
4 Wireframes (Low-Fidelity-Entwurf)	12
4.1 Auftragsliste (Dashboard)	12
4.2 Auftrag erfassen (Formular) und Detailansicht.....	13
4.3 Aktionen und Druckansicht	15
5 Einleitung und Abgrenzung	17
5.1 Eingesetzte Technologien.....	17
6 Architekturüberblick	17
6.1 Schichten des Backends.....	17
7 Domänenmodell und Statusmaschine.....	19
7.1 Zustände und erlaubte Übergänge.....	19
8 Authentifizierung und Autorisierung	20
8.1 Anmeldeablauf	20
8.2 Rollen und Berechtigungen.....	20
8.3 Sicherheitskonfiguration.....	21
9 Zentrale Fehlerbehandlung	21
10 Datenbank und Persistenz	22
10.1 Tabellen und Beziehungen	22
10.2 Sicherheit der Zugangsdaten.....	22
10.3 Anzeige-Auftragsnummer	22
11 REST-API-Referenz	23
11.1 Authentifizierung	23
11.2 Aufträge.....	23
11.3 Kunden und Mitarbeiter.....	23
11.4 Beispiel: Anmeldung	23
12.1 Kommunikation mit dem Backend	24
13 Tests und Inbetriebnahme	24

13.1 Tests.....	24
13.2 Lokale Inbetriebnahme	24
13.3 Fazit.....	24

Teil 1

Analyse und Entwurf

1 Beschreibung der Anwendung

Die Glauser Illnau AG ist ein Sanitärunternehmen, das neben Neu- und Umbauprojekten auch Reparatur- und Wartungsarbeiten im Kundenauftrag ausführt. Die hier beschriebene Webanwendung bildet den internen Ablauf eines Serviceauftrags vollständig ab – von der Annahme bis zur Verrechnung.

Bei der Auftragsannahme wird ein digitales Auftragsblatt erstellt, das den gesamten Prozess begleitet. Jeder Auftrag durchläuft dabei mehrere Zustände, die jeweils von einer bestimmten Rolle (Geschäftsleiter/Administration, Bereichsleiter, Mitarbeiter) verantwortet werden.

1.1 Prozessablauf

- **Geschäftsleiter / Administration:** Auftrag annehmen (i. d. R. telefonisch).
- **Bereichsleiter:** Auftrag disponieren (einem Mitarbeiter zuweisen, ev. Termin festlegen).
- **Mitarbeiter:** Auftrag ausführen und rapportieren.
- **Bereichsleiter:** Ausführung administrativ überwachen.
- **Bereichsleiter:** Rapport prüfen und zur Verrechnung freigeben.
- **Administration:** Auftrag verrechnen.

1.2 Funktionale Anforderungen

- Auftrag erfassen
- Auftragsdokument generieren / drucken
- Liste der Aufträge nach Zuständen einsehen
- Auftrag disponieren
- Auftrag als ausgeführt markieren
- Auftrag als verrechnet markieren

1.3 Architektur, Technologien und Schnittstellen

- **Frontend:** React (TypeScript) für clientseitige Validierung der Benutzereingaben und kontextuelles Feedback.
- **Backend:** Spring Boot 4 / Spring Framework 7, REST-Schnittstelle, Datenzugriff via Spring Data JPA.
- **Datenbank:** PostgreSQL, gehostet bei Supabase.

1.4 Technologieentscheidung: React vs. Thymeleaf

Für das Frontend standen zwei Ansätze zur Auswahl: ein serverseitig gerendertes Template-Modell mit Thymeleaf (vollständig in Spring Boot integriert) oder eine eigenständige Single-Page-Anwendung (SPA) mit React und TypeScript, die über eine REST-Schnittstelle mit dem Backend kommuniziert. Beide Varianten sind fachlich tragfähig; die Entscheidung fiel bewusst zugunsten von React.

Ausschlaggebend waren die geforderte clientseitige Validierung der Benutzereingaben mit unmittelbarem, kontextuellem Feedback sowie die klare Trennung von Präsentation und Geschäftslogik. Mit React lässt sich die Statusmaschine im UI ohne vollständiges Neuladen

der Seite abbilden (statusabhängige Aktions-Buttons, Filter nach Zustand), während das zustandslose REST-Backend unverändert wiederverwendbar bleibt – etwa für eine spätere mobile App. Thymeleaf wäre bei jeder Interaktion auf einen Server-Roundtrip mit erneutem Seitenrendering angewiesen und würde Präsentation und Backend stärker koppeln. Der Mehraufwand eines separaten Build-Schritts (Vite) und der API-Schicht wird durch bessere Wartbarkeit, Testbarkeit und Erweiterbarkeit aufgewogen.

2 OO-Modellierung

2.1 UML-Zustandsdiagramm (Navigation)

Das Zustandsdiagramm zeigt die Zustände, die ein Auftrag durchläuft, sowie die Übergänge und die jeweils auslösende Rolle. Ein abgelehnter Rapport führt den Auftrag zurück in den Zustand „Disponiert“.

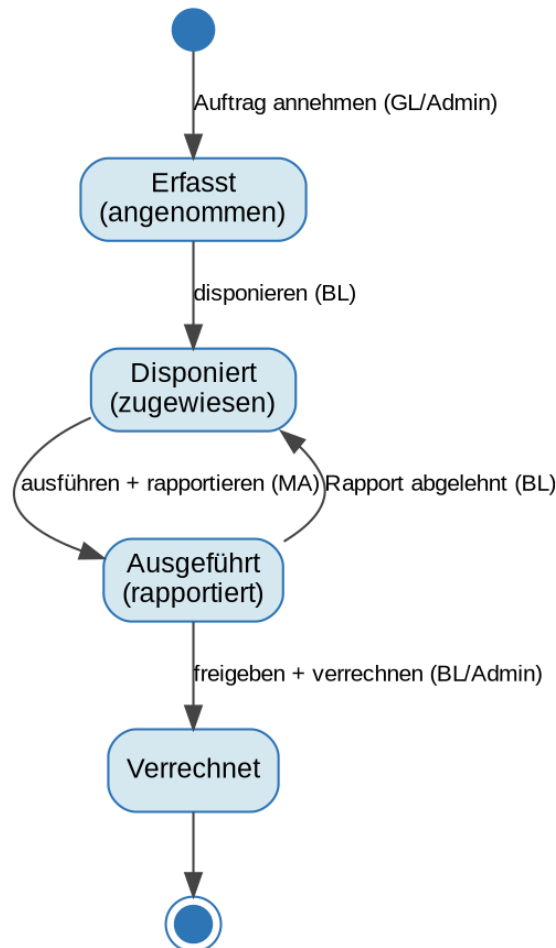


Abb. 1: Zustandsdiagramm eines Auftrags

2.2 Anwendungsfalldiagramm (Use-Cases & Aktoren)

Aktoren sind Geschäftsleiter/Administration, Bereichsleiter und Mitarbeiter. Das Diagramm ordnet die funktionalen Anforderungen den jeweiligen Aktoren zu.

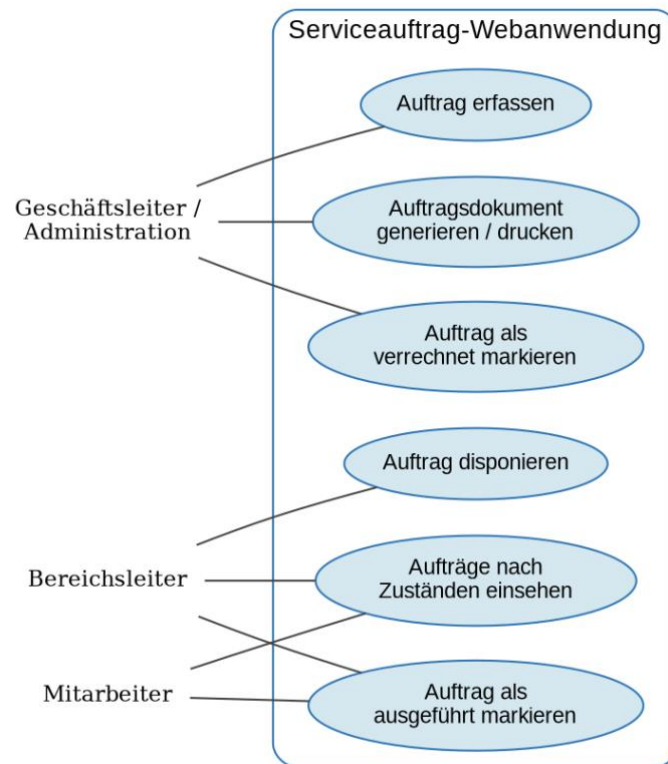


Abb. 2: Anwendungsfalldiagramm

2.3 Klassendiagramm

Das Klassendiagramm bildet die fachlichen Entitäten und ihre Beziehungen ab. Der Status eines Auftrags und die Rolle eines Mitarbeiters sind als Enumerationen modelliert.

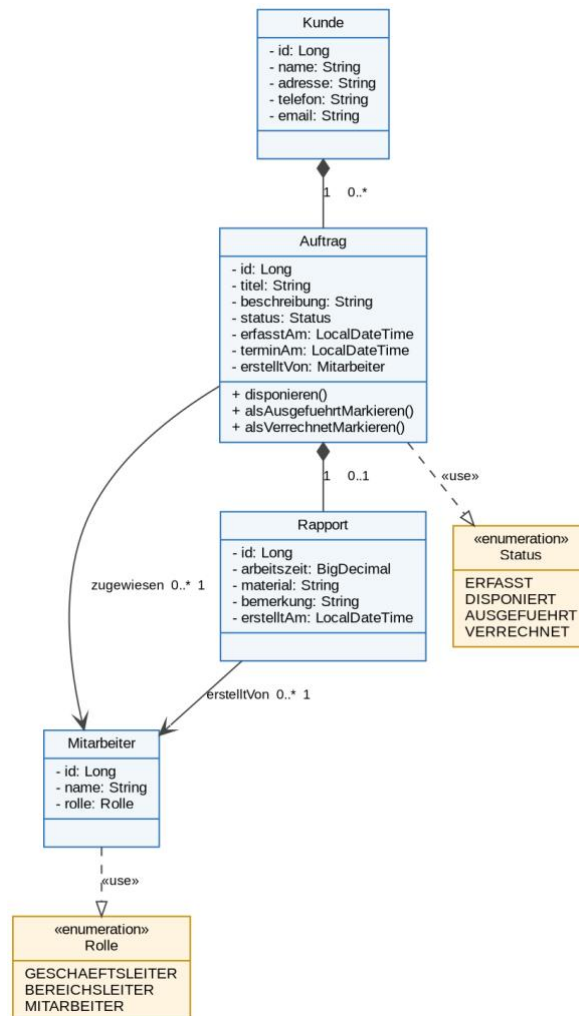


Abb. 3: Klassendiagramm

3 ERM / ERD für die PostgreSQL-Datenbank

Das Entity-Relationship-Diagramm zeigt die Tabellen, ihre Primär- und Fremdschlüssel sowie die Beziehungen. Ein Kunde kann mehrere Aufträge haben (1:n); ein Auftrag ist genau einem Mitarbeiter zugewiesen und besitzt höchstens einen Rapport (1:1).

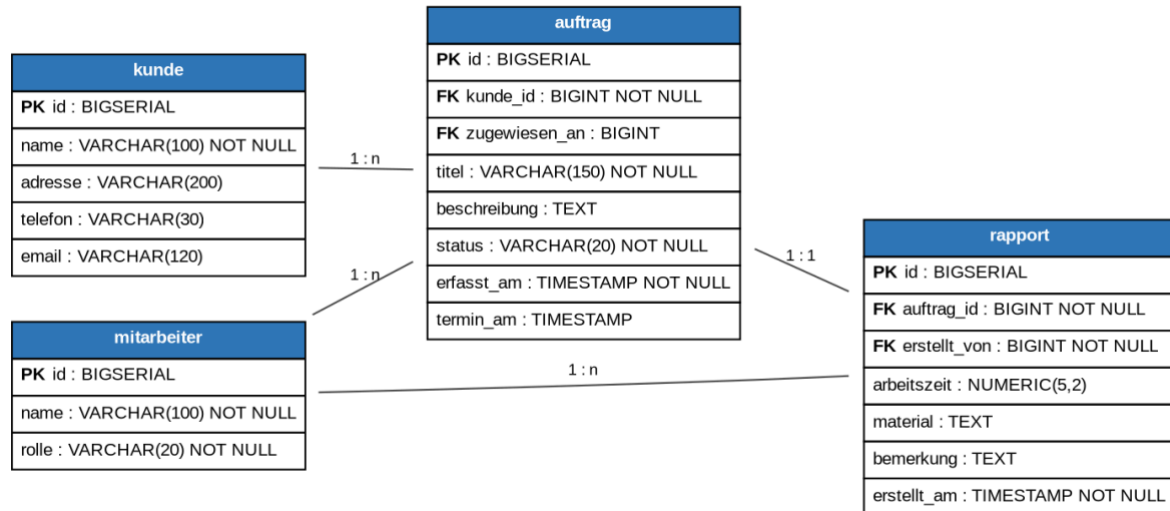


Abb. 4: Entity-Relationship-Diagramm

3.1 Verfeinertes Datenmodell (Attribute, Datentyp, Wertebereich)

Tabelle: kunde

Attribut	Datentyp	Wertebereich / Bemerkung
id	BIGSERIAL	Primärschlüssel, > 0, auto-increment
name	VARCHAR(100)	NOT NULL
adresse	VARCHAR(200)	Strasse, PLZ, Ort
telefon	VARCHAR(30)	Format frei, ev. +41...
email	VARCHAR(120)	gültige E-Mail-Adresse

Tabelle: mitarbeiter

Attribut	Datentyp	Wertebereich / Bemerkung
id	BIGSERIAL	Primärschlüssel, > 0
name	VARCHAR(100)	NOT NULL
rolle	VARCHAR(20)	NOT NULL: GESCHAEFTSLEITER BEREICHSLEITER MITARBEITER
email	VARCHAR(120)	NOT NULL, UNIQUE – Login-Kennung
passwort_hash	VARCHAR(100)	BCrypt-Hash, nie Klartext

Tabelle: auftrag

Attribut	Datentyp	Wertebereich / Bemerkung
id	BIGSERIAL	Primärschlüssel, > 0

Attribut	Datentyp	Wertebereich / Bemerkung
kunde_id	BIGINT	FK → kunde.id, NOT NULL
zugewiesen_an	BIGINT	FK → mitarbeiter.id, NULL bis disponiert
titel	VARCHAR(150)	NOT NULL
beschreibung	TEXT	optional
status	VARCHAR(20)	NOT NULL: ERFASST DISPONIERT AUSGEFUEHRT VERRECHNET
erfasst_am	TIMESTAMP	NOT NULL, Zeitpunkt der Annahme
termin_am	TIMESTAMP	optional, geplanter Termin

Tabelle: rapport

Attribut	Datentyp	Wertebereich / Bemerkung
id	BIGSERIAL	Primärschlüssel, > 0
auftrag_id	BIGINT	FK → auftrag.id, NOT NULL, UNIQUE (1:1)
erstellt_von	BIGINT	FK → mitarbeiter.id, NOT NULL
arbeitszeit	NUMERIC(5,2)	Stunden, >= 0
material	TEXT	verbrauchtes Material
bemerkung	TEXT	optional
erstellt_am	TIMESTAMP	NOT NULL

4 Wireframes (Low-Fidelity-Entwurf)

Die folgenden Wireframes skizzieren den Benutzerablauf der internen Webanwendung als Low-Fidelity-Entwurf für die Desktop-Ansicht. Sie zeigen die vier zentralen Bildschirme und deren Navigation: von der Auftragsliste über das Erfassungsformular und die Detailansicht bis zum druckbaren Auftragsdokument.

4.1 Auftragsliste (Dashboard)

Einstiegsbildschirm mit filterbarer Auftragsliste. Der Zustandsfilter (Alle, Erfasst, Disponiert, Ausgeführt, Verrechnet) blendet die Aufträge nach Status ein. Jede Zeile ist klickbar und öffnet die Detailansicht; der Status wird als Badge dargestellt.

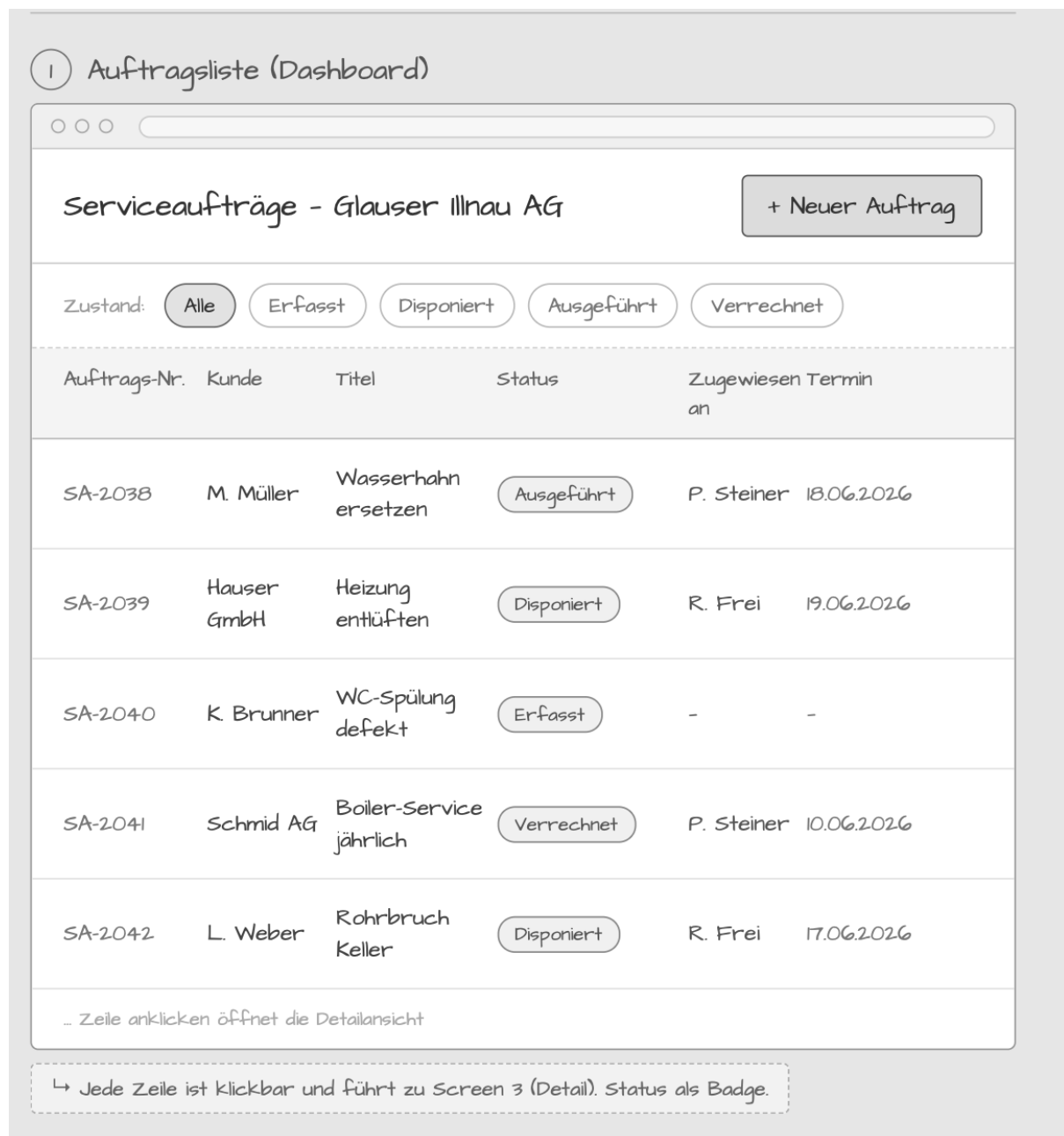


Abb. 5: Wireframe – Auftragsliste / Dashboard

4.2 Auftrag erfassen (Formular) und Detailansicht

Das Erfassungsformular enthält die Pflichtfelder Kunde, Titel und Beschreibung sowie einen optionalen Wunschtermin. Pflichtfelder (*) werden vor dem Speichern clientseitig validiert. Die Detailansicht fasst Kundendaten, Auftragsdetails und – sofern vorhanden – den Rapport zusammen.

2

Auftrag erfassen (Formular)

Neuen Auftrag erfassen

Kunde *

Kunde auswählen...

Titel *

z. B. Wasserhahn ersetzen

Beschreibung *

Mehrzeilige Beschreibung...

Gewünschter Termin (optional)

TT.MM.JJJJ

Speichern

Abbrechen

↳ * = Pflichtfeld · clientseitige Validierung vor dem Speichern

3

Auftrag-Detail

SA-2038 Ausgeführt

Wasserhahn ersetzen - Küche

KUNDENDATEN

Name

M. Müller

Adresse

Dorfstrasse 12, 8308 Illnau

Telefon

052 000 00 00

E-Mail

mueller [at] example.ch

AUFTRAGSDETAILS

Beschreibung

Alten Mischer demontieren, neuen montieren...

Erfasst am

12.06.2026

Termin

18.06.2026, vormittags

Mitarbeiter

P. Steiner

RAPPORT

nur sichtbar wenn ausgeführt

Arbeitszeit

2.5 h

Material

1x Mischbatterie, Dichtungen

Bemerkung

Eckventil leicht verkalkt, gereinigt.

Aktionen (je nach Status):

Disponieren

Als ausgeführt markieren

Als verrechnet markieren

Auftragsdokument drucken ↗

↳ Buttons erscheinen je nach Status: Erfasst→Disponieren, Disponiert→ausgeführt, Ausgeführt→verrechnet.

Abb. 6: Wireframe – Formular „Auftrag erfassen“ und Auftrag-Detail

4.3 Aktionen und Druckansicht

Im Detail erscheinen statusabhängige Aktions-Buttons (Disponieren, Als ausgeführt markieren, Als verrechnet markieren). Über „Auftragsdokument drucken“ gelangt man zur A4-optimierten Druckansicht; der Drucken-Button selbst liegt ausserhalb des Druckbereichs (window.print).

4 Auftragsdokument / Druckansicht

Symbolleiste (nicht im Druckbereich)

Drucken

Logo

Glauser Illnau AG
Sanitär · Heizung
Musterstrasse 1, 8308 Illnau
052 000 00 00

Serviceauftrag
Auftrags-Nr.: SA-2038
Datum: 17.06.2026

KUNDE

M. Müller
Dorfstrasse 12, 8308 Illnau
052 000 00 00 · mueller [at] example.ch

AUFTRAGSBESCHREIBUNG

Wasserhahn ersetzen - Küche. Alten Mischer demonstrieren, neuen montieren.
Dichtheit prüfen.

MITARBEITER

P. Steiner

TERMIN

18.06.2026, vormittags

RAPPORT

Arbeitszeit	2.5 h
Material	1x Mischbatterie, Dichtungen
Bemerkung	Eckventil leicht verkalkt, gereinigt.

Unterschrift Kunde

Unterschrift Monteur

↳ A4-Optik: „Drucken“-Button liegt ausserhalb des Druckbereichs (window.print).

Abb. 7: Wireframe – statusabhängige Aktionen und Auftragsdokument / Druckansicht

Teil 2

Implementierung

5 Einleitung und Abgrenzung

Dieser zweite Teil der Projektdokumentation beschreibt die technische Umsetzung der zuvor modellierten Webanwendung. Während Teil 1 die fachlichen Anforderungen, die OO-Modellierung, das Datenmodell und die Wireframes festlegt, dokumentiert Teil 2 die konkrete Implementierung: den Aufbau des Spring-Boot-Backends, die Durchsetzung der Statusmaschine, die Authentifizierung und Autorisierung über JSON Web Tokens, das relationale Datenbankschema sowie das React-Frontend.

Ziel der Anwendung ist es, den vollständigen Lebenszyklus eines Serviceauftrags – von der Erfassung über die Disposition und Ausführung bis zur Verrechnung – digital abzubilden. Die fachlichen Regeln werden serverseitig durchgesetzt, sodass ungültige Zustandsübergänge oder fehlende Berechtigungen unabhängig vom Client zuverlässig abgewiesen werden.

5.1 Eingesetzte Technologien

Schicht	Technologie	Zweck
Frontend	React 19, TypeScript, Vite	Benutzeroberfläche, clientseitige Validierung
Backend	Spring Boot 4.1 / Spring Framework 7	REST-API, Geschäftslogik, Sicherheit
Persistenz	Spring Data JPA / Hibernate	Objekt-relationale Abbildung
Datenbank	PostgreSQL (Supabase)	Dauerhafte Speicherung
Sicherheit	Spring Security, JWT (jjwt 0.12)	Authentifizierung, Autorisierung
Build	Maven (Wrapper), Java 25	Abhängigkeiten, Kompilierung

Tab. 1: Technologie-Stack der Anwendung

6 Architekturüberblick

Die Anwendung folgt einer klassischen Mehrschichtarchitektur. Das React-Frontend kommuniziert ausschliesslich über eine zustandslose REST-Schnittstelle mit dem Backend. Jede Anfrage an einen geschützten Endpunkt trägt ein JWT im Authorization-Header. Das Backend ist seinerseits in klar getrennte Schichten gegliedert, die jeweils eine eindeutige Verantwortung tragen.

6.1 Schichten des Backends

Paket	Verantwortung
web	REST-Controller: nehmen HTTP-Anfragen entgegen, validieren Eingaben (@Valid) und delegieren an die Service-Schicht.
service	Geschäftslogik: Statusmaschine, Berechtigungsprüfung (@PreAuthorize), Transaktionssteuerung.
repository	Datenzugriff über Spring Data JPA (JpaRepository).
domain	JPA-Entitäten (Kunde, Mitarbeiter, Auftrag, Rapport) und Enumerationen (Status, Rolle).
dto	Datenübertragungsobjekte für Anfragen und Antworten; entkoppeln die API von den Entitäten.

Paket	Verantwortung
security	JWT-Erzeugung und -Prüfung, Security-Konfiguration, Benutzer-Principal.
exception	Fachliche Ausnahmen und zentrale Fehlerbehandlung (@RestControllerAdvice).

Tab. 2: Paketstruktur des Spring-Boot-Backends

Der Datenfluss einer typischen schreibenden Anfrage verläuft entlang dieser Schichten: Der Controller empfängt das DTO, die Service-Schicht prüft Berechtigung und Statusübergang, das Repository persistiert die Änderung, und das Ergebnis wird wieder als DTO zurückgegeben.

```

Client → Controller (web) → Service (service) → Repository (repository) → PostgreSQL
      ↑ DTO (Request)           ↑ @PreAuthorize / Statusprüfung           ↓ Entität
      └────────────────────────────────────────────────────────────────────────┘
                                   DTO (Response)

```

Abb. 9: Datenfluss einer schreibenden Anfrage durch die Schichten

7 Domänenmodell und Statusmaschine

Die fachlichen Entitäten entsprechen dem Klassendiagramm aus Teil 1. Den Kern der Geschäftslogik bildet die Statusmaschine des Auftrags: Sie legt fest, welche Zustandsübergänge fachlich erlaubt sind, und wird vollständig serverseitig durchgesetzt.

7.1 Zustände und erlaubte Übergänge

Ein Auftrag durchläuft vier Zustände. Die erlaubten Übergänge sind im Enum `Status` hinterlegt; jeder andere Übergang löst eine `UngueltigerStatusUebergangException` aus (HTTP 409 Conflict).

Von	Nach	Auslösende Rolle	Bedeutung
ERFASST	DISPONIERT	Bereichsleiter	Mitarbeiter und Termin zuweisen
DISPONIERT	AUSGEFUEHRT	Mitarbeiter / Bereichsleiter	Ausführung + Rapport erfassen
AUSGEFUEHRT	VERRECHNET	Bereichsleiter / Geschäftsleiter	Rapport freigeben, verrechnen
AUSGEFUEHRT	DISPONIERT	Bereichsleiter	Rapport abgelehnt – zurück zur Disposition
VERRECHNET	–	–	Endzustand, keine Übergänge

Tab. 3: Zustandsübergänge der Auftrags-Statusmaschine

Die Übergänge sind direkt im Enum modelliert. Dadurch lässt sich die Regel an einer einzigen Stelle pflegen und sowohl beim Statuswechsel als auch in Tests wiederverwenden:

```
public Set<Status> erlaubteUebergaenge() {
    return switch (this) {
        case ERFASST    -> Set.of(DISPONIERT);
        case DISPONIERT -> Set.of(AUSGEFUEHRT);
        case AUSGEFUEHRT -> Set.of(VERRECHNET, DISPONIERT);
        case VERRECHNET -> Set.of();
    };
}

public boolean darfWechselnNach(Status ziel) {
    return erlaubteUebergaenge().contains(ziel);
}
```

Listing 1: `Status.java` – die Statusmaschine als Enum

Im `AuftragService` wird vor jedem Wechsel die Hilfsmethode `pruefeUebergang` aufgerufen. Sie wirft bei einem unzulässigen Übergang eine fachliche Ausnahme, sodass kein Auftrag jemals in einen ungültigen Zustand geraten kann:

```
private void pruefeUebergang(Auftrag auftrag, Status ziel) {
    if (!auftrag.getStatus().darfWechselnNach(ziel)) {
        throw new UngueltigerStatusUebergangException(auftrag.getStatus(), ziel);
    }
}
```

Listing 2: Serverseitige Durchsetzung des Statuswechsels

8 Authentifizierung und Autorisierung

Die Anwendung arbeitet zustandslos: Anstelle einer Server-Session weist sich ein Benutzer bei jeder Anfrage mit einem JSON Web Token (JWT) aus. Das hat den Vorteil, dass das Backend horizontal skalierbar bleibt und keine Sitzungsdaten halten muss.

8.1 Anmeldeablauf

1. Der Client sendet E-Mail und Passwort an `POST /api/auth/login`.
2. Der AuthenticationManager prüft die Daten gegen den BCrypt-Hash in der Datenbank.
3. Bei Erfolg erzeugt der JwtService ein signiertes Token mit Subject (E-Mail), Mitarbeiter-ID und Rolle.
4. Der Client speichert das Token und sendet es fortan im Header `Authorization: Bearer <token>`.
5. Der JwtAuthenticationFilter prüft bei jeder Anfrage das Token und setzt den Security-Context.

Das Token wird mit HMAC-SHA256 signiert; das Secret stammt ausschliesslich aus einer Umgebungsvariable und ist nie im Quellcode hinterlegt. Die Standard-Gültigkeit beträgt acht Stunden.

```
public String generateToken(BenutzerPrincipal principal) {
    Instant now = Instant.now();
    return Jwts.builder()
        .subject(principal.getUsername())
        .claim("mid", principal.getMitarbeiterId())
        .claim("rolle", principal.getRolle().name())
        .issuedAt(Date.from(now))
        .expiration(Date.from(now.plusMillis(expirationMs)))
        .signWith(key)
        .compact();
}
```

Listing 3: JwtService – Erzeugung des signierten Tokens

8.2 Rollen und Berechtigungen

Die Autorisierung erfolgt rollenbasiert über die Annotation `@PreAuthorize` auf den Service-Methoden (aktiviert durch `@EnableMethodSecurity`). Zusätzlich wird beim Rapportieren geprüft, dass ein Mitarbeiter nur den ihm zugewiesenen Auftrag bearbeiten darf (Eigentümerschaftsprüfung).

Aktion	Endpoint	Erlaubte Rolle(n)
Auftrag erfassen	<code>POST /api/auftraege</code>	GESCHAEFTSLEITER
Disponieren	<code>PUT .../disponieren</code>	BEREICHSLSEITER
Als ausgeführt markieren	<code>PUT .../ausgefuehrt</code>	MITARBEITER, BEREICHSLSEITER
Rapport ablehnen	<code>PUT .../rapport-ablehnen</code>	BEREICHSLSEITER
Als verrechnet markieren	<code>PUT .../verrechnet</code>	BEREICHSLSEITER, GESCHAEFTSLEITER
Aufträge / Kunden lesen	<code>GET ...</code>	alle authentifizierten

Tab. 4: Rollenbasierte Zugriffsmatrix

8.3 Sicherheitskonfiguration

Die zentrale Konfiguration in SecurityConfig schaltet CSRF ab (zustandslose API), erzwingt SessionCreationPolicy.STATELESS, gibt /api/auth/** frei und verlangt für alle übrigen /api/**-Endpunkte eine gültige Authentifizierung. Passwörter werden mit BCryptPasswordEncoder gehasht – Klartext wird zu keinem Zeitpunkt gespeichert.

9 Zentrale Fehlerbehandlung

Alle Fehler werden zentral im GlobalExceptionHandler (@RestControllerAdvice) in ein einheitliches ErrorResponse-Format überführt. So erhält der Client unabhängig von der Fehlerquelle eine konsistente, auswertbare Antwort mit Zeitstempel, Statuscode und Meldung.

Ausnahme	HTTP-Status	Bedeutung
MethodArgumentNotValid	400	Eingabevalidierung fehlgeschlagen (mit Feldfehlern)
IllegalArgumentException	400	Sonstige ungültige Anfrage
BadCredentialsException	401	E-Mail oder Passwort falsch
AccessDeniedException	403	Fehlende Berechtigung
NotFoundException	404	Ressource nicht gefunden
UngueltigerStatusUebergang	409	Unzulässiger Statuswechsel

Tab. 5: Abbildung fachlicher Ausnahmen auf HTTP-Statuscodes

10 Datenbank und Persistenz

Das in Teil 1 entworfene Datenmodell wird in PostgreSQL (gehostet bei Supabase) umgesetzt. Das Schema wird per SQL-Skript angelegt; Hibernate verwaltet das Schema bewusst nicht selbst (ddl-auto: none), damit die Datenbankstruktur kontrolliert und versioniert bleibt.

10.1 Tabellen und Beziehungen

- kunde – Auftraggeber. Ein Kunde kann mehrere Aufträge haben (1:n).
- mitarbeiter – Personen mit Rolle und Login (E-Mail + BCrypt-Hash).
- auftrag – Serviceauftrag mit Fremdschlüsseln auf Kunde und (optional) Mitarbeiter.
- rapport – genau ein Rapport pro Auftrag (1:1, durch UNIQUE-Constraint erzwungen).

Die referentielle Integrität wird über Fremdschlüssel und CHECK-Constraints auf den Enum-Spalten (status, rolle) abgesichert. Zwei Indizes auf der Tabelle auftrag beschleunigen die häufigen Filter nach Status und Kunde:

```
CREATE INDEX idx_auftrag_status ON auftrag(status);  
CREATE INDEX idx_auftrag_kunde ON auftrag(kunde_id);
```

Listing 4: Indizes für häufige Abfragen

10.2 Sicherheit der Zugangsdaten

Sämtliche Zugangsdaten – Datenbank-URL, Benutzer, Passwort sowie das JWT-Secret – werden ausschliesslich über Umgebungsvariablen geladen (\${SUPABASE_DB_URL}, \${JWT_SECRET} usw.). Im Repository liegen keine Geheimnisse; die .gitignore schliesst .env-Dateien konsequent aus.

10.3 Anzeige-Auftragsnummer

Für die Anzeige im Frontend wird aus der technischen id eine sprechende Auftragsnummer abgeleitet (z. B. SA-2038). Die Logik liegt im DTO AuftragResponse:

```
public static String anzeigeNr(Long id) {  
    return "SA-" + (2037 + id);  
}
```

Listing 5: Ableitung der Anzeige-Auftragsnummer

11 REST-API-Referenz

Die folgende Übersicht fasst alle Endpunkte zusammen. Alle Pfade unter `/api/` (ausser `/api/auth`) erfordern ein gültiges Bearer-Token. Anfrage- und Antwortkörper sind JSON.

11.1 Authentifizierung

Methode	Pfad	Beschreibung
POST	<code>/api/auth/login</code>	Anmeldung, liefert JWT + Benutzerdaten

Tab. 6: Authentifizierungs-Endpunkt

11.2 Aufträge

Methode	Pfad	Beschreibung
GET	<code>/api/auftraege</code>	Liste, optional <code>?status=...</code>
GET	<code>/api/auftraege/{id}</code>	Detailansicht inkl. Kunde, Mitarbeiter, Rapport
POST	<code>/api/auftraege</code>	Auftrag erfassen → ERFASST
PUT	<code>/api/auftraege/{id}/disponieren</code>	→ DISPONIERT
PUT	<code>/api/auftraege/{id}/ausgefuehrt</code>	Rapport erfassen → AUSGEFUEHRT
PUT	<code>/api/auftraege/{id}/rapport-ablehnen</code>	→ zurück auf DISPONIERT
PUT	<code>/api/auftraege/{id}/verrechnet</code>	→ VERRECHNET

Tab. 7: Auftrags-Endpunkte

11.3 Kunden und Mitarbeiter

Methode	Pfad	Beschreibung
GET	<code>/api/kunden</code>	Kundenliste
POST	<code>/api/kunden</code>	Kunde anlegen
GET	<code>/api/mitarbeiter</code>	Mitarbeiterliste (für Dispositions-Dropdown)

Tab. 8: Stammdaten-Endpunkte

11.4 Beispiel: Anmeldung

Anfrage:

```
POST /api/auth/login
Content-Type: application/json

{ "email": "bl@glauter.ch", "passwort": "test1234" }
```

Antwort (200 OK):

```
{
  "token": "eyJhbGciOiJIUzI1Ni...",
  "mitarbeiterId": 2,
  "name": "R. Frei",
  "email": "bl@glauter.ch",
  "rolle": "BEREICHSLLEITER"
}
```

Listing 6: Beispiel eines Login-Aufrufs

12 Frontend (React)

Das Frontend ist mit React 19 und TypeScript auf Basis von Vite umgesetzt. Es übernimmt die clientseitige Validierung der Benutzereingaben sowie kontextuelles Feedback und kommuniziert über die REST-API mit dem Backend. Die clientseitige Validierung ersetzt dabei nicht die serverseitige Prüfung, sondern ergänzt sie zugunsten einer besseren Benutzererfahrung. Die in Kapitel 4 gezeigten Wireframes bilden die Vorlage für die vier Hauptansichten.

12.1 Kommunikation mit dem Backend

- Nach dem Login wird das JWT clientseitig gehalten und jeder API-Anfrage als Bearer-Token mitgegeben.
- Die Auftragsliste lässt sich nach Status filtern; die Detailansicht zeigt Kunde, zugewiesenen Mitarbeiter und – falls vorhanden – den Rapport.
- Statusabhängige Aktionen (disponieren, ausführen, verrechnen) werden je nach Rolle des angemeldeten Benutzers ein- oder ausgeblendet.

Während der Entwicklung erlaubt die CORS-Konfiguration des Backends Anfragen vom Vite-Entwicklungsserver (<http://localhost:5173>). Auch diese Herkunft wird über eine Umgebungsvariable gesteuert und ist damit für die Produktion anpassbar.

13 Tests und Inbetriebnahme

13.1 Tests

Für Integrationstests kommt eine In-Memory-Datenbank (H2 im PostgreSQL-Modus) zum Einsatz, sodass keine externen Zugangsdaten nötig sind. Das Testprofil erzeugt das Schema aus den Entitäten (`ddl-auto: create-drop`). Spring Security stellt mit `spring-boot-starter-security-test` Hilfsmittel bereit, um authentifizierte Anfragen zu simulieren.

13.2 Lokale Inbetriebnahme

Die folgenden Schritte starten Backend und Frontend lokal:

1. Umgebungsvariablen setzen: `SUPABASE_DB_URL`, `SUPABASE_DB_USER`, `SUPABASE_DB_PASSWORD`, `JWT_SECRET`.
2. Datenbankschema und Testdaten einspielen: `db/schema.sql` und `db/seed.sql`.
3. Backend starten: `./mvnw spring-boot:run` (Port 8080).
4. Frontend starten: `npm install && npm run dev` (Port 5173).

Alle Test-Logins der Seed-Daten verwenden das Passwort `test1234`. Damit lassen sich die drei Rollen (Geschäftsleiter, Bereichsleiter, Mitarbeiter) unmittelbar ausprobieren und der vollständige Auftragsdurchlauf testen.

13.3 Fazit

Die Implementierung setzt die in Teil 1 erarbeitete Analyse vollständig um. Die zentrale fachliche Anforderung – ein kontrollierter, rollenabhängiger Auftragslebenszyklus – wird serverseitig durch die Statusmaschine und die Methoden-Sicherheit zuverlässig durchgesetzt. Durch die strikte Trennung der Schichten, die zustandslose JWT-

Authentifizierung und die konsequente Auslagerung aller Geheimnisse in Umgebungsvariablen ist die Anwendung wartbar, sicher und nachvollziehbar aufgebaut.